

# FlowerGraph — Описание протоколов передачи данных

Версия: 0.6.3.1

Целевая аудитория: программисты встроенных систем

Платформа MCU: CH32V303 / CH32V307 (и любой MCU с UART + COBS)

## Содержание

- [COM ASCII](#)
- [COM COBS](#)
- [COM mCOBS](#)
- [Сравнительная таблица](#)
- [Настройки в FlowerGraph](#)

## 1. COM ASCII

### Назначение

Простейший текстовый протокол, совместимый с PowerGraph. Используется для быстрой интеграции без бинарного парсера — достаточно `printf`.

### Формат пакета

|                                |                                |      |
|--------------------------------|--------------------------------|------|
| Канал 0 · 8 байт               | Канал N-1 · 8 байт             | CR   |
| [SIGN] [D] [D] [D] [D] [.] [D] | [SIGN] [D] [D] [D] [D] [.] [D] | 0x0D |

SIGN = '0' (0x30) — положительное значение

'-' (0x2D) — отрицательное значение

D = ASCII-цифры ('0'...'9')

Точка = всегда на 6-й позиции (фиксированный формат ±DDDDD.D)

Пробел = байт-заполнитель на 8-й позиции (0x20)

Пример, CH1=+300.0, CH2=-1234.5 (2 канала):

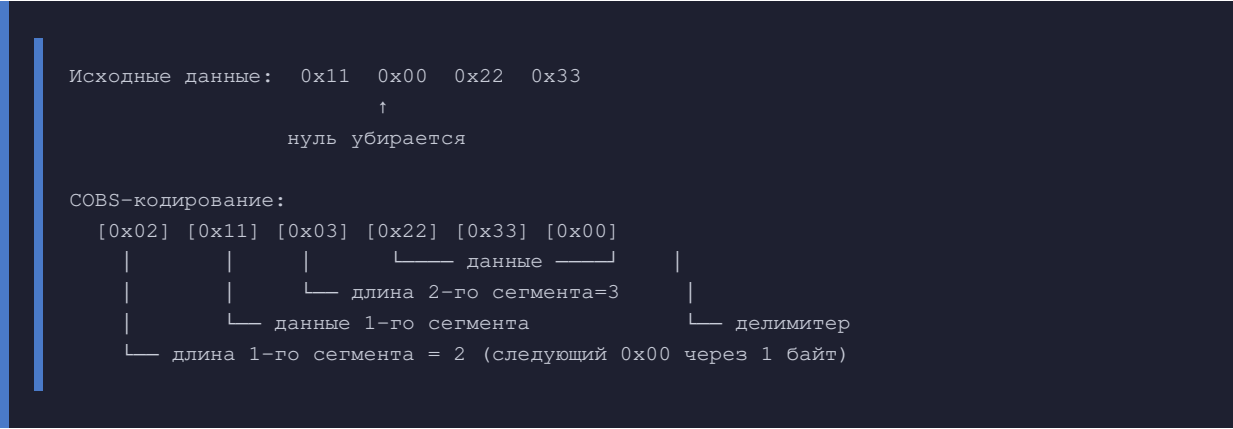
```
"0" "0" "3" "0" "0" "." "0" " " "-" "1" "2" "3" "4" "." "5" " " " " CR
30 30 33 30 30 2E 30 20 2D 31 32 33 34 2E 35 20 0D
```

### Диапазон значений

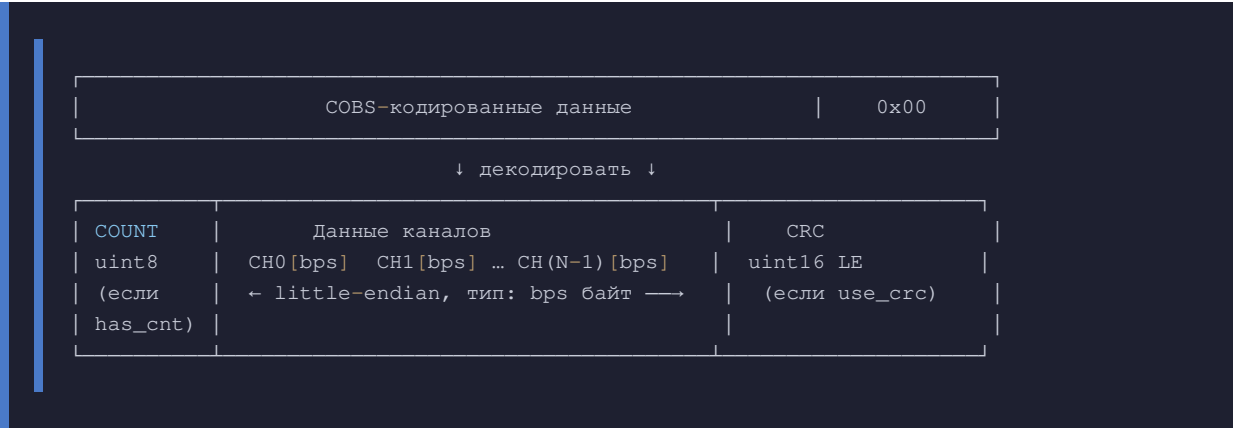
Минимум: -99999.9

Максимум: +99999.9

Точность: 0.1 (одна цифра после точки)



Структура пакета



Параметры (настраиваются в FlowerGraph)

| Параметр    | Значения                                  | Описание                               |
|-------------|-------------------------------------------|----------------------------------------|
| has_count   | true / false                              | Первый байт — wrapping-счётчик 0...255 |
| data_format | int16 / uint16 / int32 / uint32 / float32 | Тип данных каналов                     |
| use_crc     | true / false                              | CRC-16/CCITT в конце пакета            |

Байт на канал (bps)

| Тип     | bps | Диапазон                          | Примечание                    |
|---------|-----|-----------------------------------|-------------------------------|
| int16   | 2   | −32 768 ... +32 767               | Классический формат (default) |
| uint16  | 2   | 0 ... 65 535                      |                               |
| int32   | 4   | −2 147 483 648 ... +2 147 483 647 |                               |
| uint32  | 4   | 0 ... 4 294 967 295               | Совместимо с OtherSoft        |
| float32 | 4   | IEEE 754                          | Прямые физические величины    |

CRC-16/CCITT



Покрывает: [COUNT (если есть)] + [все байты данных]  
Порядок: little-endian (CRC\_L, CRC\_H)

C-реализация:

```
uint16_t crc16_ccitt(const uint8_t *data, size_t len) {
    uint16_t crc = 0xFFFF;
    for (size_t i = 0; i < len; i++) {
        crc ^= (uint16_t)data[i] << 8;
        for (int j = 0; j < 8; j++)
            crc = (crc & 0x8000) ? (crc << 1) ^ 0x1021 : (crc << 1);
    }
    return crc;
}
```

## Формулы расчёта длины пакета

```
raw_len = has_count*1 + N_CH * bps + use_crc*2
enc_len ≈ raw_len + 2          (COBS overhead + 0x00 делimiters)
pkt_rate = baudrate / (enc_len * 10)    [пакетов/с, UART 8N1]
```

## Код прошивки (C, int16, COUNT+CRC)

```
#include "cobs.h"                // nanocobs или аналог

#define N_OF_VARS    4
#define COBS_USE_CRC 1           // совпадает с настройкой FlowerGraph

typedef struct __attribute__((packed)) {
    uint8_t  count;               // wrapping 0..255
    int16_t  vals[N_OF_VARS];
#ifdef COBS_USE_CRC
    uint16_t crc;                // CRC-16/CCITT, little-endian
#endif
} PgPacket_t;

static uint8_t g_pkt_counter = 0;

void uart_send_cobs(int16_t *adc_vals) {
    PgPacket_t pkt;
    pkt.count = g_pkt_counter++;
    for (int i = 0; i < N_OF_VARS; i++)
        pkt.vals[i] = adc_vals[i];
#ifdef COBS_USE_CRC
    pkt.crc = crc16_ccitt((uint8_t*)&pkt, offsetof(PgPacket_t, crc));
#endif

    uint8_t buf[sizeof(PgPacket_t) + 2];
    size_t enc_len;
    cobs_encode(&pkt, sizeof(pkt), buf, sizeof(buf), &enc_len);
    // nanocobs уже добавил 0x00 в buf[enc_len-1]
```

```

    UART_DMA_Send(buf, enc_len);
}

```

## Код прошивки (C, float32, без COUNT, без CRC)

```

// Совместимо с прошивкой типа OtherSoft/COBS_V307
// Настройки FG: has_count=OFF, use_crc=OFF, data_format=float32

typedef struct __attribute__((packed)) {
    float vals[N_OF_VARS]; // IEEE 754, little-endian
} PgFloat_t;

void uart_send_float(float *vals) {
    PgFloat_t pkt;
    for (int i = 0; i < N_OF_VARS; i++)
        pkt.vals[i] = vals[i];

    uint8_t buf[sizeof(PgFloat_t) + 2];
    size_t enc_len;
    cobs_encode(&pkt, sizeof(pkt), buf, sizeof(buf), &enc_len);
    UART_DMA_Send(buf, enc_len);
}

```

## Производительность (460 800 бод, N=4)

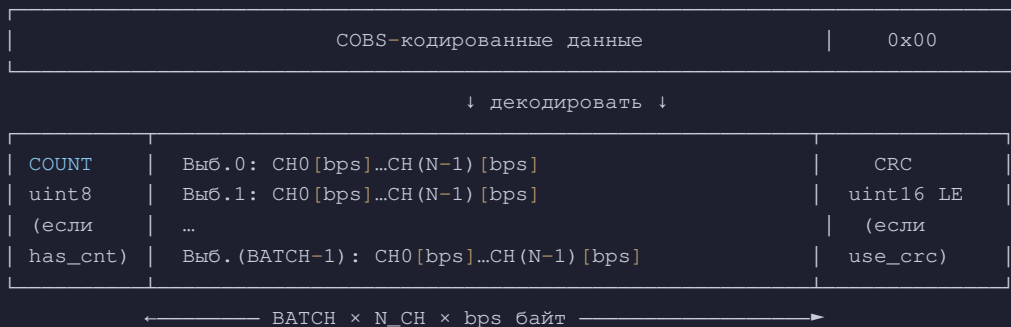
| has_count | use_crc | data_format | raw байт | enc байт | Выборки/с |
|-----------|---------|-------------|----------|----------|-----------|
| ✓         | ✓       | int16       | 11       | 13       | ~3 545    |
| ✓         | ✗       | int16       | 9        | 11       | ~4 189    |
| ✗         | ✗       | int16       | 8        | 10       | ~4 608    |
| ✓         | ✓       | float32     | 19       | 21       | ~2 194    |
| ✓         | ✗       | float32     | 17       | 19       | ~2 425    |
| ✗         | ✗       | float32     | 16       | 18       | ~2 560    |
| ✗         | ✗       | uint32      | 16       | 18       | ~2 560    |

## 3. COM mCOBS

### Назначение

Расширение COBS: один пакет содержит **BATCH** выборки подряд. Снижает удельный overhead заголовка и повышает пропускную способность при высоких частотах дискретизации.

### Структура пакета



**Порядок байт внутри данных:** выборки следуют по времени, каналы внутри выборки — по номеру.

```
offset = has_count * 1
val[sample][ch] = raw[ offset + (sample * N_CH + ch) * bps ]
```

## Параметры (дополнительно к COBS)

| Параметр                                                   | Значения | Описание                                           |
|------------------------------------------------------------|----------|----------------------------------------------------|
| batch_size                                                 | 1...64   | BATCH: число выборок в одном пакете (N_COBS_BATCH) |
| + все параметры COM COBS (has_count, data_format, use_crc) |          |                                                    |

## Автоопределение BATCH на стороне PC

FlowerGraph определяет BATCH из первого пакета:

```
payload = rlen - has_count*1 - use_crc*2
batch   = payload / (N_CH * bps)
```

Если  $\text{payload} \% (\text{N\_CH} * \text{bps}) \neq 0$  — пакет считается ошибочным.

## Код прошивки (C, int16, COUNT+CRC, BATCH=4)

```
#define N_OF_VARS      4
#define N_COBS_BATCH 4      // должно совпадать с настройкой FG
                          // (FG автоопределяет, но рекомендуется явно)

#define COBS_USE_CRC 1

// Рекомендуется DMA double-buffer:
// пока один буфер передаётся DMA, в другой пишем следующий батч
static int16_t g_samples[N_COBS_BATCH][N_OF_VARS];
static uint8_t g_batch_idx = 0;
```

```

static uint8_t g_pkt_counter = 0;
static volatile uint8_t g_dma_ready = 1;

// Вызывается из таймера с частотой sample_rate
void TIM_IRQHandler(void) {
    // Читаем АЦП в текущий слот батча
    for (int i = 0; i < N_OF_VARS; i++)
        g_samples[g_batch_idx][i] = ADC_Read(i);

    if (++g_batch_idx < N_COBS_BATCH)
        return;    // батч ещё не заполнен

    g_batch_idx = 0;

    if (!g_dma_ready) {
        g_overflow_cnt++; // DMA занят — этот батч пропускаем
        return;
    }

    // Формируем пакет в двойном буфере
    build_mcobs_packet(g_samples, g_dma_buf[g_cur]);
    g_dma_ready = 0;
    DMA_Start(g_dma_buf[g_cur], PKT_LEN, dma_done_cb);
    g_cur ^= 1;
}

void dma_done_cb(void) { g_dma_ready = 1; }

void build_mcobs_packet(int16_t samples[N_COBS_BATCH][N_OF_VARS],
                        uint8_t *out_enc) {
    typedef struct __attribute__((packed)) {
        uint8_t    count;
        int16_t    data[N_COBS_BATCH][N_OF_VARS];
#ifdef COBS_USE_CRC
        uint16_t   crc;
#endif
    } PgMPkt_t;

    PgMPkt_t pkt;
    pkt.count = g_pkt_counter++;
    memcpy(pkt.data, samples, sizeof(pkt.data));
#ifdef COBS_USE_CRC
    pkt.crc = crc16_ccitt((uint8_t*)&pkt, offsetof(PgMPkt_t, crc));
#endif
    size_t enc_len;
    cobs_encode(&pkt, sizeof(pkt), out_enc, PKT_ENC_MAX, &enc_len);
}

```

## Рекомендуемый расчёт BATCH

$$\text{batch\_opt} = \frac{\text{baudrate}}{\text{sample\_rate} \times (\text{hdr} + \text{ftr} + N \times \text{bps}) \times 10}$$

Где  $\text{hdr} = \text{has\_count} ? 1 : 0$ ,  $\text{ftr} = \text{use\_crc} ? 2 : 0$ .

Округлив в меньшую сторону, результат  $\leq 1$ . При `batch_size = 1` поведение идентично COM COBS.

Производительность (460 800 бод, N=4, int16, COUNT+CRC)

| BATCH | raw байт | enc байт | Пакетов/с | Выборок/с | Overhead |
|-------|----------|----------|-----------|-----------|----------|
| 1     | 11       | 13       | 3 545     | 3 545     | 27%      |
| 2     | 19       | 21       | 2 194     | 4 389     | 16%      |
| 4     | 35       | 37       | 1 246     | 4 983 ★   | 9%       |
| 8     | 67       | 69       | 667       | 5 334     | 4%       |
| 16    | 131      | 133      | 346       | 5 530     | 2%       |

★ — рекомендуемый BATCH для большинства задач.

Детектирование потерь пакетов (COUNT)

```
expected = (prev_count + 1) & 0xFF
if (received != expected):
    lost_pkts = (received - expected) & 0xFF
    # lost_samples = lost_pkts * batch
```

Rollover 255 → 0 обрабатывается корректно (модульная арифметика mod 256).

4. Сравнительная таблица

| Параметр                      | COM ASCII      | COM COBS                          | COM mCOBS   |
|-------------------------------|----------------|-----------------------------------|-------------|
| Кодирование                   | Текст          | COBS                              | COBS        |
| Делимитер кадра               | CR (0x0D)      | 0x00                              | 0x00        |
| Выборок в пакете              | 1              | 1                                 | 1...64      |
| Тип данных                    | float ±99999.9 | int16/uint16/int32/uint32/float32 | то же       |
| Байт COUNT                    | ✗              | опционально                       | опционально |
| CRC-16/CCITT                  | ✗              | опционально                       | опционально |
| Детект потерь                 | ✗              | при COUNT                         | при COUNT   |
| Совместимость PowerGraph      | ✓              | ✗                                 | ✗           |
| Макс. выборки/с (N=4, 460800) | ~1 400         | ~4 600                            | ~5 500      |
| Сложность прошивки            | ☆☆☆            | ★★☆                               | ★★★         |

5. Настройки в FlowerGraph

Диалог COM COBS / COM mCOBS

Открыть: Панель инструментов → Источник: COM COBS → ⚙️ Настройка

| Поле          | Что задаёт                                               |
|---------------|----------------------------------------------------------|
| Порт          | COM-порт (COM1...COMn)                                   |
| Скорость      | Baudrate (таблица точности для CH32V303 @ 144 МГц)       |
| Каналов       | 0 = авто-определение по первому пакету                   |
| Байт COUNT    | ✓ — первый байт пакета является счётчиком                |
| Тип данных    | Тип и размер каждого значения канала                     |
| CRC-16/CCITT  | ✓ — последние 2 байта пакета являются контрольной суммой |
| Выборок/пакет | (только mCOBS) N_COBS_BATCH                              |

Диаграмма «Структура пакета»

В диалоге отображается интерактивная диаграмма, обновляющаяся при изменении любого параметра. Каждое поле показывает тип данных и занимаемые байты.

Важно: MCU и FlowerGraph должны совпадать

Все три параметра (has\_count, data\_format, use\_crc) должны быть одинаково настроены на стороне MCU и в FlowerGraph. Несоответствие приведёт к тому, что все пакеты будут отбрасываться как ошибочные.

Приложение: наборы зависимостей

nanocobs (Charles-Nielsen, Unlicense)

```
https://github.com/nicowilllis/nanocobs
Совместимость с FlowerGraph: полная
API: cobs_encode(src, src_len, dst, dst_max, &dst_len)
      → 0x00 делимитер уже добавлен в dst, не нужно добавлять вручную
```

Точность baudrate CH32V303 @ APB2 = 144 МГц

| Baudrate  | Ошибка | Примечание                      |
|-----------|--------|---------------------------------|
| ≤ 230 400 | 0%     | Стандартный ряд RS-232          |
| 460 800   | 0.16%  | Ряд 1.8432 МГц × 2 <sup>n</sup> |
| 921 600   | 0.16%  |                                 |
| 1 843 200 | 0.16%  |                                 |